

netmon

netmon

Passive network monitor for auditing what your device leaks to the ISP and which sites it contacts. Captures on all interfaces and logs timestamped, categorized JSONL.

What it records

File	Contents
<code>dns.jsonl</code>	Every DNS query (name, type, resolver), every answer (A/AAAA/CNAME/... with TTL), and every HTTPS/SVCB (type 65/64) record with its SvcParams: <code>alpn</code> , <code>port</code> , <code>ipv4hint</code> , <code>ipv6hint</code> , and an <code>ech</code> flag
<code>tls.jsonl</code>	SNI from the ClientHello of every HTTPS connection — TCP TLS and decrypted QUIC Initials (<code>transport</code>), the cleartext <code>alpn</code> list, with <code>ech: true</code> marking a cover name (see Limitations)
<code>http.jsonl</code>	Plaintext HTTP requests: method, path, Host, User-Agent; known captive-portal probes carry <code>tag: "captive-portal"</code>
<code>flows.jsonl</code>	Every new connection: protocol, direction, local/remote IP+port, service guess, hostname (reverse-mapped from observed DNS answers), scope (<code>internet</code> , <code>lan</code> , or <code>multicast</code>), and a <code>note</code> on disclosive services (NTP, STARTTLS mail)
<code>summary.json</code>	Written on exit: top DNS names, top SNI hostnames, top internet hosts, event counts

All events carry ISO 8601 UTC timestamps with millisecond precision. Each run writes to `logs/run-<stamp>/`.

Run

Capture needs raw-socket privileges:

```
sudo $(command -v uv) run netmon.py           # all interfaces
sudo $(command -v uv) run netmon.py -i wlan0  # one interface
sudo $(command -v uv) run netmon.py --bpf 'not port 22' -q
```

Or grant the capability once and drop sudo:

```
sudo setcap cap_net_raw+eip "$(readlink -f .venv/bin/python3)"
uv run netmon.py
```

`-q/--quiet` suppresses per-event stdout (files are always written). A stats line is logged every 30 s. Stop with Ctrl-C — the summary is written on exit.

Reading the results

What your ISP sees even with HTTPS: everything in `dns.jsonl` (unless you use encrypted DNS), every `sni` value in `tls.jsonl`, and every remote IP in `flows.jsonl`. Quick looks:

```
jq -r '.sni' logs/run-*/tls.jsonl | sort | uniq -c | sort -rn
# sites visited via SNI
jq -r 'select(.kind=="dns_query") | .qname' logs/run-*/dns.jsonl |
sort -u
jq -r 'select(.scope=="internet") | .hostname // .remote_ip' logs/
run-*/flows.jsonl | sort | uniq -c | sort -rn
```

Mitigating what the ISP sees: [MITIGATIONS.md](#). Operations: [RUNBOOK.md](#).

Limitations

- QUIC (HTTP/3, `udp/443`): the SNI in a QUIC Initial packet is **not** hidden from your ISP. Initial packets are protected with keys derived from the connection's cleartext Destination Connection ID plus a public, version-specific salt (RFC 9001 §5.2), so any passive observer on the path can decrypt the ClientHello — and mainstream DPI already does. netmon decrypts these Initials (QUIC v1 and v2, reassembling the CRYPTO stream across coalesced and fragmented packets) and logs the SNI in `tls.jsonl` with `"transport": "quic"`. It parses only `udp/443` and only the client's Initial flight.
- Encrypted DNS (DoH/DoT/DoQ) does hide the query name from your ISP; it shows up here only as flows to the resolver.
- Encrypted Client Hello (ECH) hides the SNI by design: the cleartext ClientHello then carries a public cover name, not the

site you visited. netmon flags these with `"ech": true` in `tls.jsonl` — such a line means the real hostname was hidden from your ISP (a leak prevented), and the `sni` value is only the cover name. If the ECH handshake fails, the browser retries without it and the real SNI goes out in cleartext, which netmon still records.

- **Segmented parsing:** SNI and HTTP requests are reassembled across TCP segments (post-quantum ClientHellos routinely span several), so a request split mid-header is still parsed once whole. The one gap is a connection whose opening segment was never captured — if netmon starts mid-stream, that flow is not reassembled and its SNI/HTTP is skipped (the flow itself is still logged).

What this tool does NOT show you

Even at zero of qname/SNI/remote-IP (ECH + encrypted DNS + VPN), an on-path ISP still has signal netmon cannot capture — the honest bound on “what the ISP sees”:

- **Traffic analysis** — packet sizes, timing, direction, and per-flow byte volumes. These alone fingerprint individual websites (and often individual pages) behind TLS, ECH, and a VPN, because the shape of a page load is distinctive. netmon logs that a flow happened, not its size/timing profile.
- **TLS client fingerprint (JA3/JA4)** — the ordering of cipher suites, extensions, and supported groups in the ClientHello identifies the client software and version. netmon reads SNI and ALPN from that hello but does not compute the fingerprint.
- **TLS 1.2 server certificate** — on the still-common TLS 1.2 path the server’s certificate (with its SANs) crosses the wire in cleartext during the handshake; TLS 1.3 encrypts it. netmon does not parse server certificates.
- **IPv6 address leakage** — a SLAAC EUI-64 address embeds the NIC MAC, and a stable IPv6 identifies the device across networks even as the SNI is hidden. netmon records the addresses but does not flag this derivation.

Only Tor-class tooling (onion routing plus traffic padding) meaningfully addresses the traffic-analysis/correlation channel; a VPN or ECH relocates the observer, it does not remove the shape of your traffic.